Simulacros de exámenes de Programación en Python – 2.º DAW

Modelo A

Instrucciones:

- El examen consta de 20 preguntas.
- Cada pregunta tiene tres respuestas posibles.
- Solamente una respuesta es correcta.
- Marca una única opción por pregunta.

1. ¿Qué valor muestra el siguiente programa?

```
def restar(x, y):
    return x - y

print(restar(10, 4))
```

A) 14

B) -6

C) 6

2. ¿Qué muestra por pantalla el siguiente programa?

```
def doble(n):
    print(n * 2)

resultado = doble(5)
print(resultado)
```

A) Primero 10 y después None

B) Primero 10 y después 10

C) Solo 10, porque la última línea no imprime nada

3. ¿Qué imprime este programa?

```
total = 100

def aplicar_descuento(precio):
    total = precio - 10
    return total

aplicar_descuento(50)
print(total)
```

A) 40

B) 100

C) NameError

4. El fichero `operaciones.py` contiene la función `sumar(a, b)`. ¿Qué línea de `main.py` permite llamar directamente a `sumar(2, 3)`?

A) `from operaciones import sumar`
B) `import sumar from operaciones`
C) `from sumar import operaciones`

5. Completa la función para que, si `b` vale 0, devuelva el texto indicado sin intentar dividir:

```
def dividir(a, b):
    if b == 0:
        ______
    return a / b
```

A) `print('Error: división entre cero')`
B) `b = 1`
C) `return 'Error: división entre cero'`

6. En la sala de cine, `'L'` es libre y `'X'` ocupada. ¿Qué imprime este código?

```
sala = [['L', 'L', 'L'],
        ['L', 'X', 'L']]
print(sala[1][1])
```

A) X
B) L
C) IndexError

7. La matriz `sala` tiene 8 filas y 10 butacas por fila. El usuario elige la fila 3, butaca 5 (numeración que empieza en 1). ¿Qué instrucción accede a esa butaca?

A) `sala[3][5]`
B) `sala[2][4]`
C) `sala[4][2]`

8. ¿Qué valor imprime este programa?

```
sala = [['X', 'L', 'X'],
        ['L', 'X', 'X'],
        ['L', 'L', 'X']]
contador = 0
for i in range(len(sala)):
    for j in range(len(sala[i])):
        if sala[i][j] == 'X' and i != j:
            contador += 1
print(contador)
```

A) 5
B) 3
C) 2

9. Completa la función para que reserve la butaca solo si está libre:

```
def reservar(sala, fila, columna):
    if sala[fila][columna] == 'L':
        ______
        return True
    return False
```

A) `sala[fila][columna] == 'X'`

B) `sala[fila] = 'X'`

C) `sala[fila][columna] = 'X'`

10. ¿Qué ocurre al ejecutar este código?

```
sala = []
for i in range(3):
    sala.append(['L'] * 4)

sala[2][4] = 'X'
```

A) Se reserva la última butaca de la última fila

B) IndexError, porque las columnas válidas son los índices 0 a 3

C) IndexError, porque las filas válidas son solo los índices 0 y 1

11. Dado el siguiente código, ¿qué son `v1` y `v2`?

```
class Vehiculo:
    def __init__(self, matricula):
        self.matricula = matricula

v1 = Vehiculo('1234ABC')
v2 = Vehiculo('5678DEF')
```

A) Dos instancias de la clase Vehiculo

B) Dos clases independientes con el mismo constructor

C) Dos métodos que reciben una matrícula

12. ¿Qué imprime este programa?

```
class Plaza:
    def __init__(self, numero):
        self.numero = numero
        self.ocupada = False

    def ocupar(self):
        self.ocupada = True

p = Plaza(7)
p.ocupar()
print(p.numero, p.ocupada)
```

A) 7 False

B) 7 True

C) 0 True

13. Al ejecutar `p.mostrar()`, ¿a qué se refiere `self` dentro del método `mostrar`?

```
class Parking:
    def __init__(self):
        self.plazas = 20

    def mostrar(self):
        print(self.plazas)

p = Parking()
p.mostrar()
```

A) Al objeto concreto sobre el que se ejecuta el método (`p`)

B) A la clase Parking en general, compartida por todos los objetos

C) Al fichero donde se define la clase

14. Dada la clase `Ticket`, ¿qué instrucción muestra correctamente el importe del objeto `t`?

```
class Ticket:
    def __init__(self, minutos):
        self.minutos = minutos

    def importe(self):
        return self.minutos * 0.5

t = Ticket(90)
```

A) `print(Ticket.importe())`

B) `print(importe.t())`

C) `print(t.importe())`

15. ¿Qué imprime este programa?

```
class Parking:
    def __init__(self):
        self.plazas = [None] * 3

    def entrada(self, matricula):
        for i in range(len(self.plazas)):
            if self.plazas[i] is None:
                self.plazas[i] = matricula
                return i
        raise Exception('Parking completo')

p = Parking()
p.entrada('AAA')
p.entrada('BBB')
p.entrada('CCC')
try:
    p.entrada('DDD')
except Exception as e:
    print('Error:', e)
```

A) Error: Parking completo

B) Error: DDD

C) No imprime nada, porque DDD se guarda en la plaza 0

16. ¿Qué tipo de error produce este código?

```
precio = 5
print('Total:' precio)
```

A) NameError

B) TypeError

C) SyntaxError

17. El usuario escribe 45. ¿Qué error se produce y cómo se corrige?

```
minutos = input('Minutos: ')
horas = minutos / 60
```

A) ValueError, porque 45 no es un número válido

B) TypeError, porque `minutos` es texto y debe convertirse con `int()`

C) NameError, porque `horas` no está definida

18. Se espera que el programa muestre 1.5 horas, pero imprime 1. ¿Qué tipo de error es y cómo se corrige?

```
def minutos_a_horas(minutos):
    return minutos // 60

print(minutos_a_horas(90))
```

A) Error de lógica: se usa `//` (división entera) y debería usarse `/`

B) TypeError: `minutos` debe convertirse a `str` antes de dividir

C) SyntaxError: el operador `//` no existe en Python 3

19. Observa el siguiente mensaje de error. ¿Dónde está el problema y cómo se soluciona?

```
Traceback (most recent call last):
  File "main.py", line 12, in <module>
    parking.registrar_entrada('1234ABC')
  File "parking.py", line 25, in registrar_entrada
    self.plazas.add(matricula)
AttributeError: 'list' object has no attribute 'add'
```

A) En `main.py`, línea 12: la matrícula tiene un formato incorrecto

B) En `parking.py`, línea 25: las listas no tienen `add()`; hay que usar `append()`

C) En `parking.py`, línea 25: falta importar la clase `list`

20. ¿Qué imprime este código y por qué?

```
def normalizar(matricula):
    matricula.upper().strip()
    return matricula

print(normalizar(' 1234abc '))
```

A) `1234ABC`, porque `upper()` y `strip()` modifican la cadena original

B) AttributeError, porque las cadenas no tienen el método `strip()`

C) `1234abc` sin cambios, porque ambos métodos devuelven una cadena nueva que no se guarda

Modelo B

Instrucciones:

- El examen consta de 20 preguntas.
- Cada pregunta tiene tres respuestas posibles.
- Solamente una respuesta es correcta.
- Marca una única opción por pregunta.

1. ¿Qué valor devuelve la llamada a la función?

```
def potencia(base, exponente):
    return base ** exponente

print(potencia(2, 3))
```

A) 8

B) 6

C) 9

2. ¿Qué imprime este programa?

```
def operar(opcion, a, b):
    if opcion == 1:
        return a + b
    elif opcion == 2:
        return a - b
    elif opcion == 3:
        return a * b
    else:
        return 0

print(operar(2, 9, 4) + operar(5, 9, 4))
```

A) 13

B) 0

C) 5

3. ¿Qué valor imprime este programa?

```
def cuadrado(n):
    return n * n

def suma_cuadrados(a, b):
    return cuadrado(a) + cuadrado(b)

resultado = suma_cuadrados(2, cuadrado(2))
print(resultado)
```

A) 8

B) 16

C) 20

4. El fichero `operaciones.py` contiene `def sumar(a, b): return a + b`. Completa `main.py` para que se muestre 9:

```
# main.py
import operaciones

print(_____(4, 5))
```

A) `sumar`

B) `operaciones.sumar`

C) `operaciones(sumar)`

5. El usuario escribe `abc` cuando se ejecuta `leer_numero()`. ¿Qué ocurre?

```
def leer_numero():
    texto = input('Número: ')
    try:
        return float(texto)
    except ValueError:
        print('Valor no válido')
        return None
```

A) Imprime «Valor no válido» y la función devuelve `None`

B) `float('abc')` devuelve 0.0 y la función continúa

C) El programa termina con un traceback de `ValueError`

6. ¿Qué imprime este código sobre la sala de cine (1 = ocupada, 0 = libre)?

```
sala = [[0, 1, 0, 0],
        [1, 1, 0, 1],
        [0, 0, 0, 0]]
print(len(sala), len(sala[0]))
```

A) 4 3

B) 3 4

C) 12 4

7. Se quiere mostrar cada fila de la sala en una línea distinta. ¿Qué instrucción falta?

```
for fila in sala:
    for butaca in fila:
        print(butaca, end=' ')
    _____
```

A) `print(fila)`

B) `break`

C) `print()`

8. ¿Qué imprime este programa?

```
sala = [[0, 1],
        [1, 1],
        [1, 0]]
total = 0
for fila in range(3):
    if sala[fila][0] == 1:
        total += fila
print(total)
```

A) 3

B) 2

C) 4

9. ¿Qué valor imprime este programa?

```
def butacas_libres(fila):
    libres = 0
    for estado in fila:
        if estado == 0:
            libres += 1
    return libres

sala = [[0, 1, 0], [1, 1, 0]]
print(butacas_libres(sala[1]))
```

A) 2

B) 1

C) 3

10. La sala tiene 2 filas y 3 columnas. ¿Qué ocurre al ejecutar este código?

```
sala = [[0, 0, 0],
        [0, 0, 0]]
for fila in range(3):
    for columna in range(3):
        sala[fila][columna] = 1
```

A) Se rellena toda la sala sin errores

B) IndexError inmediato en la primera iteración, sin modificar nada

C) IndexError al llegar a `fila = 2`, después de rellenar las dos filas existentes

11. ¿Qué expresión lee el valor de un atributo del objeto `c`?

```
class Coche:
    def __init__(self, matricula, color):
        self.matricula = matricula
        self.color = color

    def describir(self):
        return self.matricula + ' ' + self.color

c = Coche('4455JKL', 'rojo')
```

A) `c.color`

B) `c.describir()`

C) `Coche('4455JKL', 'rojo')`

12. ¿Qué imprime este programa?

```
class Contador:
    def __init__(self):
        self.entradas = 0

    def entrar(self):
        self.entradas += 1

a = Contador()
b = Contador()
a.entrar()
a.entrar()
b.entrar()
print(a.entradas, b.entradas)
```

A) 3 3

B) 2 1

C) 1 1

13. Dada la clase, ¿qué instrucción crea un parking de 20 plazas?

```
class Parking:
    def __init__(self, capacidad):
        self.capacidad = capacidad
```

A) `Parking(capacidad) = 20`

B) `p = Parking.capacidad(20)`

C) `p = Parking(20)`

14. Completa el método para que el programa imprima 4:

```
class Parking:
    def __init__(self):
        self.plazas = [None] * 5

    def plazas_libres(self):
        libres = 0
        for plaza in self.plazas:
            if plaza is None:
                libres += 1
        ______

p = Parking()
p.plazas[0] = '1111AAA'
print(p.plazas_libres())
```

A) `return libres`

B) `print(libres)`

C) `return self.libres`

15. ¿Qué imprime este programa?

```
class Parking:
    def __init__(self):
        self.matriculas = ['1111AAA', '2222BBB']

    def salida(self, matricula):
        if matricula not in self.matriculas:
            raise ValueError('Vehículo no encontrado')
        self.matriculas.remove(matricula)
        return len(self.matriculas)

p = Parking()
print(p.salida('1111AAA'))
try:
    print(p.salida('1111AAA'))
except ValueError as e:
    print(e)
```

A) 1 y después 0

B) 1 y después Vehículo no encontrado

C) 2 y después Vehículo no encontrado

16. ¿Qué error produce este código?

```
def registrar_entrada(matricula):
    hora = 10
    return hora

registrar_entrada('1234ABC')
print(hora)
```

A) TypeError

B) AttributeError

C) NameError

17. ¿Qué error produce este código y cómo se corrige?

```
class Plaza:
    def __init__(self, numero):
        self.numero = numero

p = Plaza(3)
print(p.ocupada)
```

A) AttributeError; definir `self.ocupada = False` en `__init__`

B) NameError; definir `ocupada` antes del `print`

C) TypeError; convertir `p` a texto antes de imprimir

18. ¿Qué imprime este código y qué problema tiene?

```
def hay_plaza_libre(plazas):
    for plaza in plazas:
        if plaza is None:
            return True
        else:
            return False

print(hay_plaza_libre(['AAA', None, None]))
```

A) True, porque hay plazas libres y la función funciona bien

B) False, porque el `else` devuelve False en la primera plaza ocupada sin revisar las demás

C) None, porque el bucle nunca llega a ejecutarse

19. El fichero `parking.py` define la clase `Parking`. En `main.py` se escribe `from parking import Parqing`. ¿Qué error se produce y cómo se corrige?

A) NameError; hay que declarar la variable `Parqing` antes

B) SyntaxError; falta una coma en el `import`

C) ImportError; el nombre importado debe ser `Parking`, igual que la clase

20. El parking cobra 3 € por hora o fracción. ¿Qué imprime este código?

```
minutos = 130
horas = minutos // 60
if minutos % 60 > 0:
    horas += 1
print(horas * 3)
```

A) 9

B) 6

C) 6.5

Modelo C

Instrucciones:

- El examen consta de 20 preguntas.
- Cada pregunta tiene tres respuestas posibles.
- Solamente una respuesta es correcta.
- Marca una única opción por pregunta.

1. ¿Qué muestra este programa de la calculadora?

```
def media(a, b):
    return (a + b) / 2

print(media(7, 3))
```

A) 10
B) 5.0
C) 5

2. ¿Qué ocurre al ejecutar este código?

```
def sumar(a, b):
    print(a + b)

resultado = sumar(2, 3) + 1
```

A) Imprime 5 y luego se produce un TypeError (`None + 1`)
B) Imprime 6 y `resultado` vale 6
C) No imprime nada y `resultado` vale 1

3. ¿Qué valor imprime este programa?

```
contador = 0

def incrementar():
    global contador
    contador += 1
    return contador

incrementar()
incrementar()
print(incrementar() + contador)
```

A) 5
B) 3
C) 6

4. La función `restar(a, b)` está definida en `calculos.py`. ¿Qué línea de `main.py` permite usar `restar(8, 3)` directamente, sin prefijo?

A) `import restar from calculos`
B) `import calculos.restar`
C) `from calculos import restar`

5. Se quiere que la función devuelva la opción elegida como número entero. ¿Qué instrucción falta?

```
def pedir_opcion():
    opcion = input('Opción (1-4): ')
    while opcion not in ['1', '2', '3', '4']:
        opcion = input('Opción no válida. Repite (1-4): ')
    ______
```

A) `print(int(opcion))`
B) `return int(opcion)`
C) `opcion = int`

6. En la sala (`'L'` libre, `'O'` ocupada), ¿qué imprime este código?

```
sala = [['L', 'L'],
        ['L', 'O'],
        ['O', 'O']]
sala[0][1] = 'O'
print(sala[0])
```

A) `['L', 'O']`
B) `['O', 'L']`
C) `['L', 'L']`

7. Se quiere mostrar al usuario la fila 1 para el índice 0, la fila 2 para el índice 1, etc. ¿Qué expresión falta?

```
for i in range(len(sala)):
    print('Fila', ______, ':', sala[i])
```

A) `i`
B) `i + 1`
C) `i - 1`

8. ¿Qué imprime este programa?

```
sala = [['L', 'O', 'L'],
        ['O', 'O', 'L'],
        ['L', 'L', 'O']]
resultado = []
for i in range(3):
    if sala[i][i] == 'O':
        resultado.append(i)
print(resultado)
```

A) `[0, 1]`
B) `[0, 2]`
C) `[1, 2]`

9. La función debe devolver `True` solo si todas las butacas de la fila están ocupadas (`'O'`). ¿Qué instrucción falta?

```
def fila_llena(fila):
    for butaca in fila:
        if butaca == 'L':
            ______
    return True
```

A) `return False`

B) `return True`

C) `break`

10. La sala tiene 4 filas y 5 butacas por fila. El usuario introduce fila 4 y butaca 1 (numeración desde 1). ¿Qué ocurre?

```
sala = []
for i in range(4):
    sala.append(['L'] * 5)

fila = 4
butaca = 1
sala[fila][butaca] = 'O'
```

A) Se reserva la fila 4, butaca 1, sin errores

B) IndexError: no existe la fila de índice 4 porque no se restó 1 al valor del usuario

C) Se reserva la butaca de la fila 5, butaca 2

11. ¿Qué imprime este programa?

```
class Parking:
    def __init__(self, nombre):
        self.nombre = nombre

centro = Parking('Centro')
norte = Parking('Norte')
print(norte.nombre)
```

A) Centro

B) Parking

C) Norte

12. ¿Qué imprime este programa?

```
class Plaza:
    def __init__(self, numero):
        self.numero = numero
        self.matricula = None

    def ocupar(self, matricula):
        self.matricula = matricula

    def esta_libre(self):
        return self.matricula is None

p = Plaza(1)
p.ocupar('9999ZZZ')
print(p.esta_libre())
```

A) False

B) True

C) 9999ZZZ

13. Justo después de crear el objeto, ¿qué imprime este código?

```
class Plaza:
    def __init__(self, numero):
        self.numero = numero
        self.libre = True

p = Plaza(4)
print(p.libre)
```

A) False

B) True

C) 4

14. Completa el método para que el programa muestre 10.0:

```
class Registro:
    def __init__(self, matricula, minutos):
        self.matricula = matricula
        self.minutos = minutos

    def importe(self, tarifa_minuto):
        return ______

r = Registro('7777MMM', 40)
print(r.importe(0.25))
```

A) `minutos * tarifa_minuto`

B) `self.minutos * self.tarifa_minuto`

C) `self.minutos * tarifa_minuto`

15. ¿Qué imprime este programa?

```
class Parking:
    def __init__(self):
        self.plazas = [None, None]

    def entrada(self, matricula):
        if matricula in self.plazas:
            raise ValueError('Ya está dentro')
        posicion = self.plazas.index(None)
        self.plazas[posicion] = matricula
        return posicion

p = Parking()
print(p.entrada('AAA'))
print(p.entrada('BBB'))
try:
    p.entrada('AAA')
except ValueError as error:
    print('Error:', error)
```

A) 0, después 1 y después Error: Ya está dentro

B) 0, después 1 y después «AAA» se guarda en una tercera plaza

C) 0, después 0 y después Error: Ya está dentro

16. ¿Qué tipo de error produce este código?

```
def calcular(minutos)
    return minutos * 0.5
```

A) NameError

B) SyntaxError

C) TypeError

17. ¿Qué error se produce y por qué?

```
def calcular_importe(minutos, tarifa):
    return minutos * tarifa

print(calcular_importe(30))
```

A) SyntaxError: falta un paréntesis

B) NameError: `tarifa` no está definida fuera de la función

C) TypeError: falta el argumento `tarifa` en la llamada

18. Observa el mensaje de error. ¿Cuál es la causa más probable?

```
Traceback (most recent call last):
  File "main.py", line 8, in <module>
    print(parking.cobrar(entrada, salida))
  File "parking.py", line 31, in cobrar
    return (salida - entrada) * self.tarifa
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

A) `entrada` y `salida` son texto (`str`) y deben convertirse a número antes de restar

B) `self.tarifa` no existe y debe crearse en `__init__`

C) La función `cobrar` no se ha importado en `main.py`

19. ¿Qué tipo de error produce este código?

```
matricula = 'abc123'
print(matricula.mayusculas())
```

A) NameError

B) AttributeError

C) TypeError

20. Enunciado: los primeros 15 minutos son gratis y después se cobran 0.25 € por cada minuto adicional. ¿Qué imprime este código y es correcto?

```
def calcular_importe(minutos):
    if minutos <= 15:
        return 0
    return minutos * 0.25

print(calcular_importe(30))
```

A) 3.75; el código ya descuenta los 15 minutos gratuitos

B) 7.5; es correcto porque se cobran todos los minutos

C) 7.5; error lógico: no se restan los 15 minutos gratuitos

Modelo D

Instrucciones:

- El examen consta de 20 preguntas.
- Cada pregunta tiene tres respuestas posibles.
- Solamente una respuesta es correcta.
- Marca una única opción por pregunta.

1. ¿Qué muestra este programa?

```
def multiplicar(a, b):
    return a * b

print(multiplicar(6, 7) + 1)
```

A) 42

B) 67

C) 43

2. ¿Qué imprime este programa?

```
def calcular(opcion, a, b):
    if opcion == '+':
        return a + b
    elif opcion == '-':
        return a - b
    elif opcion == '*':
        return a * b
    else:
        return None

print(calcular('*', 4, 5), calcular('/', 4, 5))
```

A) 20 0.8

B) 20 None

C) 9 None

3. ¿Qué imprime este programa?

```
def restar(a, b):
    return a - b

x = restar(b=2, a=10)
y = restar(2, 10)
print(x, y)
```

A) 8 -8

B) -8 8

C) 8 8

4. El fichero `calculos.py` contiene la función `sumar(a, b)`. ¿Qué línea de `main.py` permite escribir `calculos.sumar(1, 2)`?

A) `import calculos`
B) `from calculos import sumar`
C) `import sumar`

5. ¿Qué ocurre al ejecutar `main.py`?

```
# calculos.py
def sumar(a, b):
    return a + b
```

```
# main.py
from calculos import sumar

print(calculos.sumar(1, 2))
```

A) Imprime 3
B) ImportError, porque `sumar` no existe en `calculos`
C) NameError, porque solo se importó `sumar` y el nombre `calculos` no existe

6. En la sala (1 = ocupada, 0 = libre), ¿qué imprime este código?

```
sala = [[0, 0, 0],
        [0, 1, 0]]
print(sala[1][0], sala[1][1])
```

A) 1 0
B) 0 0
C) 0 1

7. Completa la condición para que se muestre el mensaje cuando la butaca ya está ocupada:

```
if ______:
    print('Butaca ocupada')
else:
    sala[fila][columna] = 1
```

A) `sala[fila][columna] == 1`
B) `sala[fila][columna] = 1`
C) `sala[fila] == 1`

8. ¿Qué imprime este programa?

```
sala = [[1, 1, 0, 1],
        [1, 0, 0, 0]]
primera = None
for fila in range(2):
    for columna in range(4):
        if sala[fila][columna] == 0 and primera is None:
            primera = [fila, columna]
print(primera)
```

A) `[1, 1]`

B) `[0, 2]`

C) `[1, 3]`

9. ¿Qué imprime este programa?

```
sala = [[1, 0, 1],
        [0, 0, 0],
        [1, 1, 1]]
ocupadas_por_fila = []
for fila in sala:
    cuenta = 0
    for butaca in fila:
        cuenta += butaca
    ocupadas_por_fila.append(cuenta)
print(ocupadas_por_fila)
```

A) `[2, 1, 2]`

B) `[3, 0, 2]`

C) `[2, 0, 3]`

10. ¿Qué imprime este programa?

```
sala = [[1, 0, 1],
        [1, 1, 0],
        [0, 1, 1]]
liberadas = 0
for fila in range(3):
    if sala[fila][1] == 1:
        sala[fila][1] = 0
        liberadas += 1
print(liberadas, sala[2])
```

A) `2 [0, 0, 1]`

B) `3 [0, 0, 1]`

C) `2 [0, 1, 1]`

11. ¿Qué imprime este programa?

```
class Plaza:
    def __init__(self, numero):
        self.numero = numero

plaza_1 = Plaza(1)
plaza_2 = Plaza(2)
plaza_2.numero = 10
print(plaza_1.numero, plaza_2.numero)
```

A) 10 10

B) 1 10

C) 1 2

12. ¿Qué imprime este programa?

```
class Parking:
    def __init__(self, plazas_totales):
        self.plazas_totales = plazas_totales
        self.ocupadas = 0

    def entrar(self):
        if self.ocupadas < self.plazas_totales:
            self.ocupadas += 1
            return True
        return False

p = Parking(2)
resultados = [p.entrar(), p.entrar(), p.entrar()]
print(resultados, p.ocupadas)
```

A) `[True, True, True] 3`

B) `[True, False, False] 1`

C) `[True, True, False] 2`

13. ¿Cuántas veces aparece «Registro creado» al ejecutar este código?

```
class Registro:
    def __init__(self):
        print('Registro creado')

r1 = Registro()
r2 = Registro()
```

A) Dos veces, una por cada objeto creado

B) Una vez, porque la clase solo se define una vez

C) Ninguna, hasta que se llame a un método

14. Completa el método para que libere la plaza de la matrícula indicada:

```
class Parking:
    def __init__(self):
        self.plazas = ['1111AAA', None, '2222BBB']

    def liberar(self, matricula):
        for i in range(len(self.plazas)):
            if self.plazas[i] == matricula:
                ______
                return True
        return False
```

A) `self.plazas[i] == None`

B) `self.plazas[i] = None`

C) `plazas[i] = None`

15. ¿Qué imprime este programa?

```
class Ticket:
    def __init__(self, entrada, salida):
        if salida < entrada:
            raise ValueError('Salida anterior a la entrada')
        self.minutos = salida - entrada

def crear_ticket(entrada, salida):
    try:
        return Ticket(entrada, salida)
    except ValueError:
        return None

t1 = crear_ticket(60, 150)
t2 = crear_ticket(200, 100)
print(t1.minutos, t2)
```

A) 90 -100

B) Un ValueError sin controlar

C) 90 None

16. ¿Qué tipo de error produce este código?

```
plazas_libres = 5
print('Plazas libres: ' + plazas_libres)
```

A) TypeError

B) NameError

C) SyntaxError

17. Al ejecutar `main.py` aparece `NameError: name 'Parking' is not defined`. ¿Cuál es la corrección?

```
# parking.py
class Parking:
    def __init__(self):
        self.plazas = [None] * 20
```

```
# main.py
import parking

mi_parking = Parking()
```

A) Escribir `import Parking` en `main.py`

B) Usar `parking.Parking()` o `from parking import Parking`

C) Renombrar el fichero `parking.py` como `Parking.py`

18. ¿Qué imprime este código y qué problema tiene?

```
plazas = [None, '1234ABC', None]

def esta_ocupada(plazas, numero):
    return plazas[numero] is None

print(esta_ocupada(plazas, 1))
```

A) True; el resultado es correcto porque la plaza 1 tiene vehículo

B) None; el operador `is` devuelve `None` al comparar con cadenas

C) False; error lógico: la condición está invertida y debería ser `is not None`

19. ¿Cuál de los siguientes programas NO llega a imprimir `Inicio`?

A) `print('Inicio'); x = (3 + 4`

B) `print('Inicio'); x = 3 / 0`

C) `print('Inicio'); x = y + 4`

20. Enunciado: «hasta 15 minutos incluidos son gratuitos». La función tiene un error. ¿Qué llamada de prueba lo detecta?

```
def es_gratis(minutos):
    return minutos < 15
```

A) `es_gratis(10)`

B) `es_gratis(15)`

C) `es_gratis(20)`